

## Network Security: Authentication, SSL, Firewalling

### Lecture 26



UNIVERSITÀ DEGLI STUDI DI NAPOLI  
**FEDERICO II**



# Network Security

## Basics of Network Security

- Given the previous attack types, we can now define the set of proprieties that a **secure communication** should guarantee:
  - **Confidentiality**: only the sender and intended receiver should be able to understand the contents of the transmitted message (sniffing avoidance).
  - **Message integrity**: the content of the communication must not be altered, either maliciously or by accident.
  - **End-point authentication**: both the sender and receiver should be able to confirm the identity of the other party involved in the communication (spoofing avoidance).
  - **Operational security**: to rely on a network infrastructure that prevents malicious hosts to sneak into the communication.
- The **first 3 proprieties are software-based** the last one (operational security) typically relies on **specific hardware** (firewalls, intrusion detection systems).

# Network Security

## End-point Authentication

- The **end-point authentication** is the process of **one entity proving its identity to another entity** over a computer network.
- Typically, an **authentication protocol (AP)** would run **before the two communicating parties run some other protocol**.
  - It is used to **establish the identities** of both involved parties **before they start to work**.
  - Examples of such protocols may be a reliable data transfer protocol (based on TCP), a routing information exchange protocol (based on DV or LS), an e-mail protocol (e.g., SMTP), etc.
- To perform end-point authentication **we can rely on the certification authority (CA)** previously introduced, but this is not enough.

# Network Security

## End-point Authentication

- Let's assume that **host A needs to authenticate himself to B** before they can start to work together.
- If A and B have already communicated and the IP address of A is still the same, **B could authenticate A by checking the IP address** inside the datagram.
  - **If the address is the well-known** one, we can assume A is sending the message.
- This simple techniques of authentication **works for “naive” intruders** trying to disguise as A, but it is clearly not sufficient: a **“less naive” intruder may forge the address** into the packet.

# Network Security

## End-point Authentication

- As we have seen, it is still possible **to forge an IP datagram to insert a specific source IP address** (spoofing).
- **One way of avoiding IP spoofing is through the routers:** a router may be **configured to forward only legitimate datagrams**, i.e., containing IP source addresses that really belong to hosts.
  - The router must be aware of the IPs of hosts (otherwise, hosts would be unreachable).
  - The router can detect packets whose IP address is not consistent with that source.
  - Forged datagrams may just be dropped by the router so they can't harm anyone.
- Unfortunately, **this capability is not universally deployed** nor enforced, so we must assume spoofing as possible.

# Network Security

## End-point Authentication

- **Another approach is to use a secret password** (most common): the password works as a **shared secret** between the authenticator and the person being authenticated.
  - Gmail, Facebook, telnet, FTP, and many other services use password authentication.
  - This **shared secret can also be used for integrity check**.
- The first issue here is that **the intruder may eavesdrop the password** from messages, using both password and forged IP.
- **Eavesdropping can be avoided by encrypting the password**, so it becomes unreadable for the intruder.
  - Notice that **if we use a symmetric encryption** here, we do not need an additional password because the shared key itself (which must be a shared secret between the 2 hosts) may also work as a password.

# Network Security

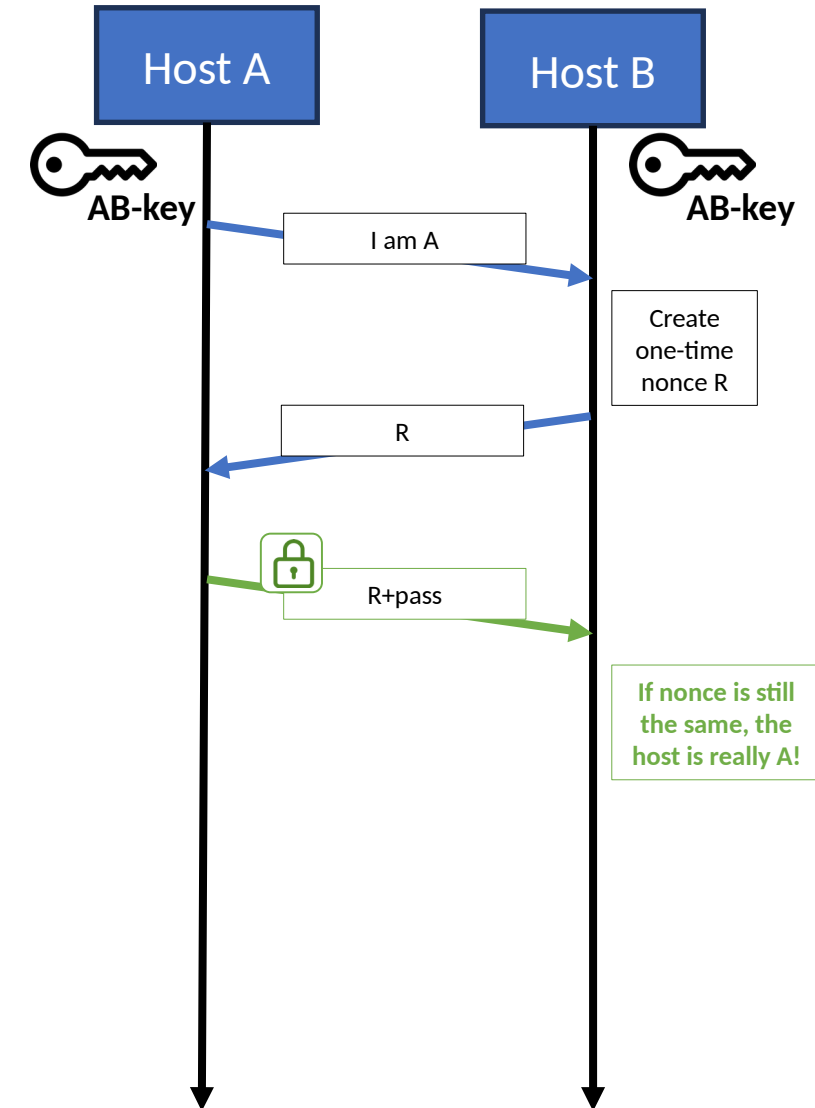
## End-point Authentication

- Encryption is good, but we are still not safe, a **“very smart” intruder may still be able to sneak into the communication** disguised as A through a **playback attack**.
- In a **playback attack** (or replay attack) the intruder tries to mimic one host:
  - The intruder sniffs packets from A to B trying to get the **encrypted version of A's password**.
  - If succeeded, the **intruder may play back the encrypted version of the password** to B in a new session, **using the password even without understanding it**.
  - Differently from standard MiM attacks (which are real-time), sniffing and playback are performed separately.
- Here the problem is that host **B is unable to tell if A was live**, i.e., if there is **an active session running** between the real A and B.
  - **This problem is somehow similar to TCP connection establishment** in which the two host use three-way handshake to ensure that both are alive on the two ends.
- **In TCP handshake hosts use random initial sequence numbers** (or very old sequence number) to avoid possible retransmissions to be interpreted as SYN/ACK segments. A **similar idea can be adopted** for authentication purposes.

# Network Security

## End-point Authentication: Nonce

- In this case we can use a **nonce**: a **random or pseudo-random number** that a protocol uses **only once in a lifetime**.
- Example: assuming A and B share a symmetric key, the **nonce** can be used as follows:
  1. Host A sends a message "I am A" to B.
  2. Host B chooses a nonce R and sends it to A.
  3. Host A encrypts the nonce along with the password using A-B symmetric secret key and sends the encrypted nonce back to B.
  4. Host B decrypts the received message. **If the decrypted nonce equals the nonce he sent A**, then A is authenticated.
- This password becomes sort of a one-time password:
  - All encrypted passwords have this nonce number with it, so they cannot be reused into other sessions (by someone else).
  - If the **password + nonce** is correctly received by B, we can reasonably assume that **there is A living behind it**.



# Network Security

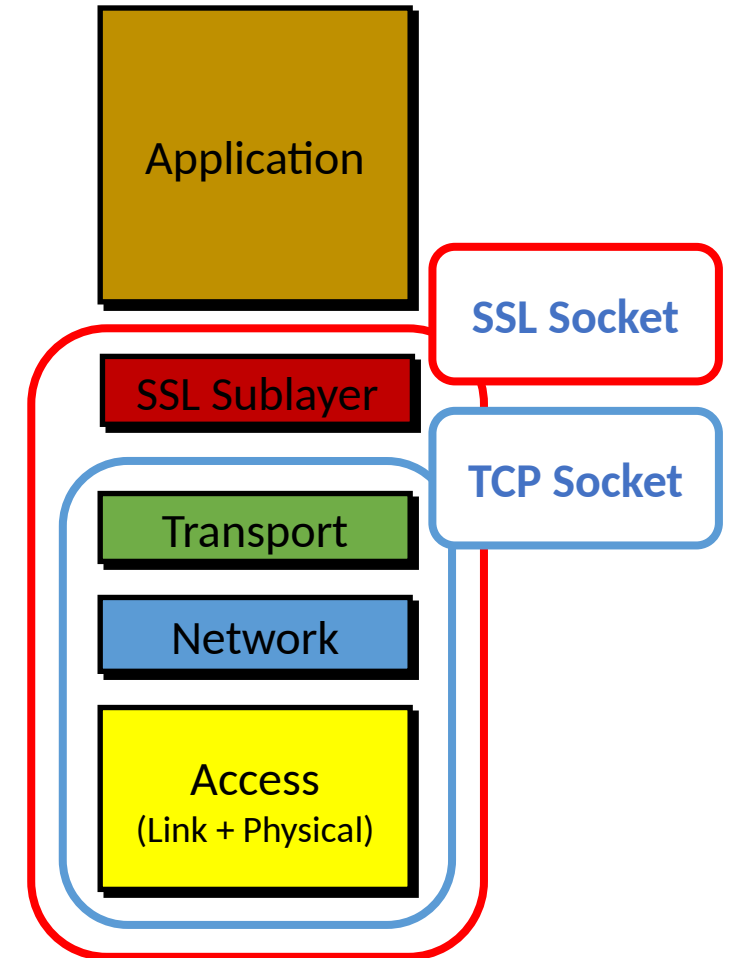
## SSL

- So far, we have seen that **several cryptography techniques can be used and integrated** to provide confidentiality, data integrity and end-point authentication to TCP connections.
- Such techniques are implemented through the **Secure Socket Layer (SSL)**, which is an **enhanced version of TCP** providing security services.
  - There is also a slightly modified version of SSL version 3, called **Transport Layer Security (TLS)**.
  - There is an equivalent for UDP-based communication called **Datagram TLS (DTLS)**.
- SSL is used by several application protocols:
  - Web, e-mail, instant messaging, voice over IP, etc.
- It is possible to notice that often the connection between our browser and a website uses **HTTPS (HTTP+SSL/TLS)** rather than HTTP.

# Network Security

## SSL

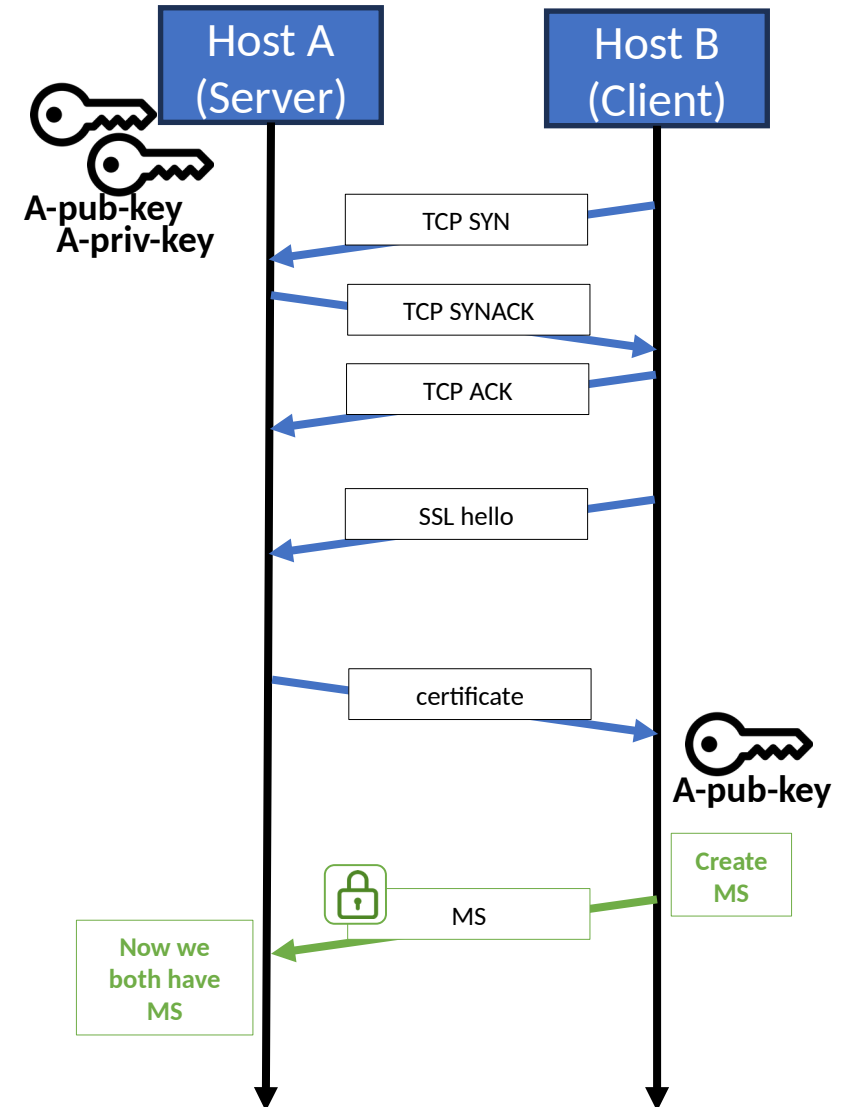
- Since SSL secures TCP, it **can be employed by any application that runs over TCP**.
- **SSL provides a simple Application Programmer Interface (API) with sockets**, which is similar and analogous to TCP's API. When an application wants to employ SSL, the application includes SSL classes/libraries.
- Although **SSL technically resides in the application layer**, from the developer's perspective **it looks like a transport protocol** that provides TCP's services enhanced with security services.
- SSL has three main phases: **handshake, key derivation, data transfer**.



# Network Security

## SSL: Handshake

- Let's assume a client host (B) wants to use SSL to communicate with a server (A). During the **SSL handshake phase**, client B needs to:
  - Establish a **TCP connection** with A.
  - Verify that **A is really A** (not a fake server).
  - Send to A a **master secret key** (shared secret).
- The process works as follows:
  - Once the TCP connection is established, **B sends to A a hello message**.
  - Server **A responds with a certificate**, which contains her public key (asymmetric).
  - Since the certificate has been certified by a CA, client **B trust that the public key in the certificate belongs to A**.
  - B then generates a Master Secret (MS)**, which will **only be used for this SSL session (nonce)**, encrypts the MS with the public key of A and sends it to A.
  - Server **A decrypts the message with the private key** to get the MS. After this phase, **only B and A know the master secret** for this SSL session.



# Network Security

## SSL: Key Derivation

- Since **MS** is a **shared secret between B and A**, it could be used as the **symmetric key** for all encryption and data integrity checking during the session.
- On the other hand, **it is safer for A and B to use different keys** for encryption and integrity checking in the two streams of data (A-to-B and B-to-A).
- **Four keys can be created** from the MS:
  - $E_{B\text{-to-}A}$  = session **encryption key** for data sent from B to A
  - $M_{B\text{-to-}A}$  = session **MAC key** for data sent from B to A
  - $E_{A\text{-to-}B}$  = session **encryption key** for data sent from A to B
  - $M_{A\text{-to-}B}$  = session **MAC key** for data sent from A to B

# Network Security

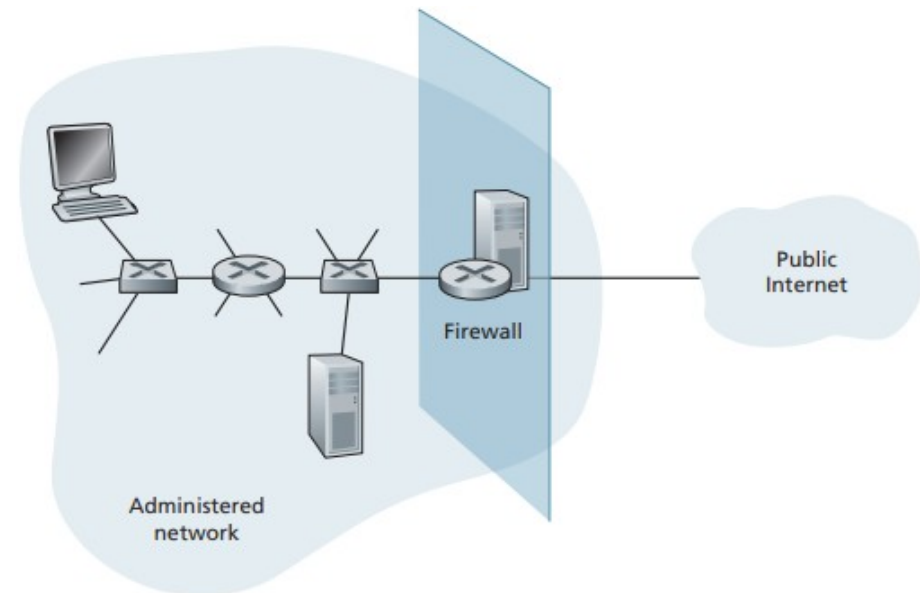
## SSL: Data Transfer

- **SSL secures TCP** by breaking the data stream (from application) into records. For each record:
  1. **SSL appends a MAC to each record** for integrity checking ( $\text{record}_i + \text{MAC}$ ).
  2. The modified **record is encrypted**.
  3. The **encrypted record is passed to TCP** for transmission.
- This process ensures data integrity of the single records, but **what about the entire stream?**
- Since only payload is encrypted, **a MiM between the A and B is still able to damage the stream by inverting or removing segments:**
  - For instance, the MiM may capture 2 consecutive segments, then invert their order along with their TCP sequence numbers, then send the 2 inverted segments to the receiver.
  - The receiver would not notice the inversion until data **arrives to the application**.
- To avoid this issue **SSL also integrates a sequence number into the message** before encryption:
  - The real **encrypted message will be  $\text{record}_i + \text{MAC} + \text{sequenceNumber}_i$** .
  - Knowing the procedure and the MAC, the **receiver would be able to certify the integrity of the message**.

# Network Security

## Firewall

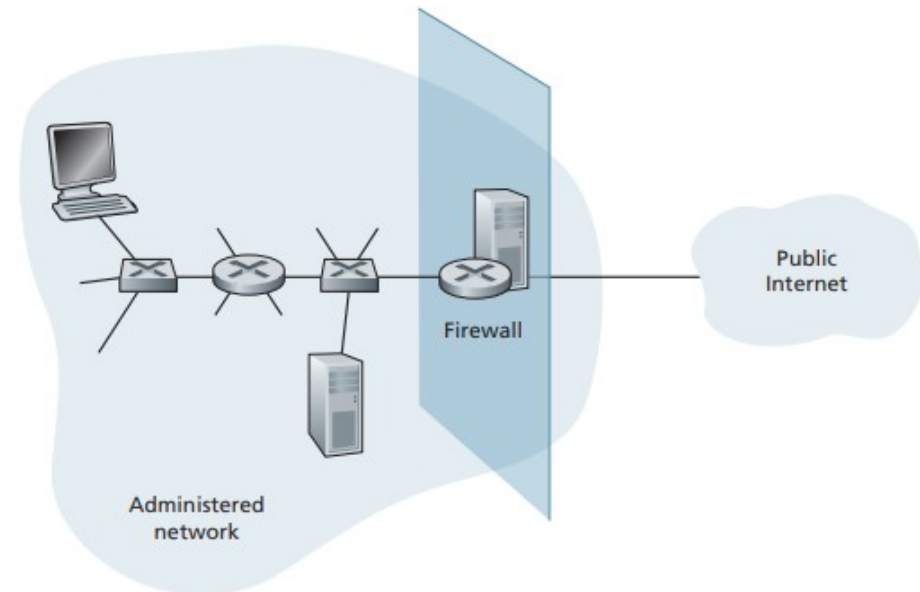
- A **firewall** is a combination of **hardware and software** that isolates (protects) a network from Internet by **allowing/denying packets** to pass.
- All **incoming/outcoming traffic should pass through the firewall** and only authorized traffic (as defined by the local security policy) is allowed to pass.
  - Firewall works as a **single access point to the public network**, but large organizations may have multiple levels or distributed firewalls.
- The firewall itself must be **immune to penetration**.
  - If compromised, it can provide a false sense of security.
- We will see 3 approaches:
  - Traditional packet filtering.
  - Stateful filtering.
  - Application gateway.



# Network Security

## Firewall: Traditional Packet Filtering

- A **packet filter** is typically implemented on the **gateway** (the router connecting local network to the ISP).
- It **examines each single datagram**, determining whether the **datagram should be accepted/dropped** depending on administrator-specific **rules**.
  - We can have specific set of rules for entering or exiting datagrams or for the single interfaces.
- **Rules** are typically based on:
  - IP source or destination address.
  - Protocol type in IP datagram field.
    - TCP, UDP, ICMP, etc.
  - Source and destination port.
  - TCP flag bits: SYN, ACK, etc.
  - ...



# Network Security

## Firewall: Traditional Packet Filtering

- A network administrator configures the firewall based on the **policy of the organization**, here some examples:
  - **Allow Web traffic only** by setting a rule that **blocks all TCP SYN** segment having destination **port different than 80**.
  - **Deny (some) streaming services** by setting a rule that **blocks non-critical UDP traffic** (often used for streaming).
  - **Deny incoming ping** by setting a rule that **blocks outgoing ICMP ping responses**.
- A filtering policy can also be based on a **combination of addresses and port numbers** (it can be IP-specific).
  - For example, a filtering router could **forward all Telnet datagrams** (port number 23) **except those going to and coming from a list of specific IP addresses**.
  - Note: such policies **do not provide protection against spoofing**.

# Network Security

## Firewall: Traditional Packet Filtering

- Firewall rules are implemented in routers with **access control lists**.
- Let's consider the example of a **simple access control list** for the interface connecting a router with the ISP.
  - We want **only web traffic** to pass.
- We can set **four rules** as follows:
  - The **first rule** allows any TCP packet with **destination port 80 to leave** the organization's network.
  - The **second rule** allows any TCP packet with **source port 80 and flag ACK = 1 to enter** the organization's network.
    - Remind: all web messages carry ACK numbers.
  - The **third and fourth rules** together allow **DNS packets to enter and leave** the organization's network.

| action | source address       | dest address         | protocol | source port | dest port | flag bit |
|--------|----------------------|----------------------|----------|-------------|-----------|----------|
| allow  | 222.22/16            | outside of 222.22/16 | TCP      | > 1023      | 80        | any      |
| allow  | outside of 222.22/16 | 222.22/16            | TCP      | 80          | > 1023    | ACK      |
| allow  | 222.22/16            | outside of 222.22/16 | UDP      | > 1023      | 53        | —        |
| allow  | outside of 222.22/16 | 222.22/16            | UDP      | 53          | > 1023    | —        |
| deny   | all                  | all                  | all      | all         | all       | all      |

# Network Security

## Firewall: Traditional Packet Filtering

- There is a problem with traditional filtering: it is **stateless**.
- Although rather restrictive, **this table allows any packet** arriving from the outside **with ACK = 1 and source port 80** to get through the filter.
- Unfortunately, such packets are known to be used in **DoS attacks**.
- The naive solution would be **to block TCP ACK packets as well**, but such approach would also **prevent internal users from surfing the Web**.

| action | source address       | dest address         | protocol | source port | dest port | flag bit |
|--------|----------------------|----------------------|----------|-------------|-----------|----------|
| allow  | 222.22/16            | outside of 222.22/16 | TCP      | > 1023      | 80        | any      |
| allow  | outside of 222.22/16 | 222.22/16            | TCP      | 80          | > 1023    | ACK      |
| allow  | 222.22/16            | outside of 222.22/16 | UDP      | > 1023      | 53        | —        |
| allow  | outside of 222.22/16 | 222.22/16            | UDP      | 53          | > 1023    | —        |
| deny   | all                  | all                  | all      | all         | all       | all      |

# Network Security

## Firewall: Stateful Packet Filtering

- **Stateful filters** solve this problem by **tracking all ongoing TCP connections** in a connection table, in order to understand if the traffic is due to a legitimate connection:
  - The **firewall recognizes the beginning of a new connection** (i.e., the SYN-SYNACK-ACK sequence of the three-way handshake), **and the end of a connection** (i.e., the FIN packet).
  - The firewall can also (conservatively) **assume that the connection is over** when it hasn't seen any activity over the connection for a certain amount of time (e.g., 60 seconds).
- A stateful table **typically includes a “check connection” column** that specifies if the packets are within an established connection.

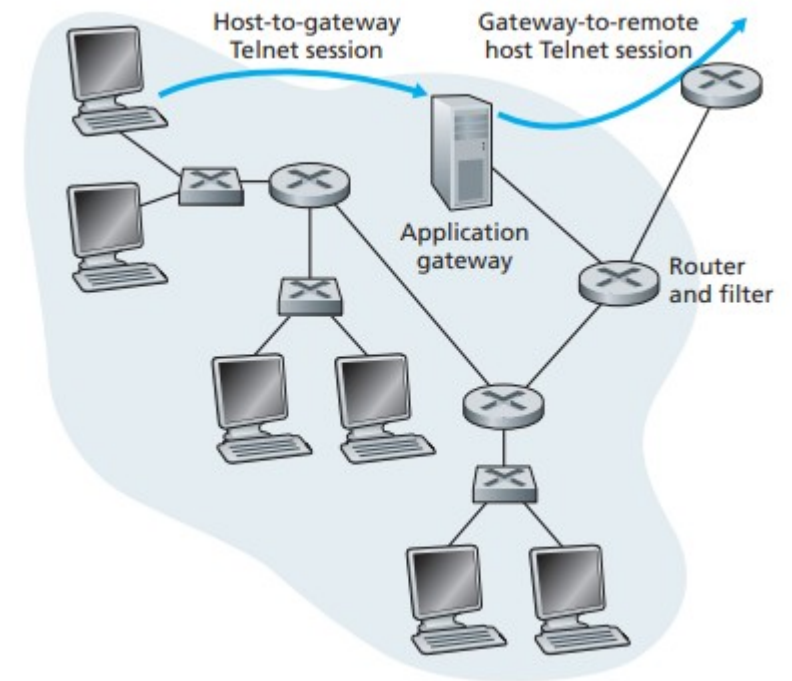
| action | source address       | dest address         | protocol | source port | dest port | flag bit | check connexion |
|--------|----------------------|----------------------|----------|-------------|-----------|----------|-----------------|
| allow  | 222.22/16            | outside of 222.22/16 | TCP      | > 1023      | 80        | any      |                 |
| allow  | outside of 222.22/16 | 222.22/16            | TCP      | 80          | > 1023    | ACK      | X               |
| allow  | 222.22/16            | outside of 222.22/16 | UDP      | > 1023      | 53        | —        |                 |
| allow  | outside of 222.22/16 | 222.22/16            | UDP      | 53          | > 1023    | —        |                 |
| deny   | all                  | all                  | all      | all         | all       | all      |                 |

In this case only ACK packets within established connections are allowed.

# Network Security

## Firewall: Application Gateway

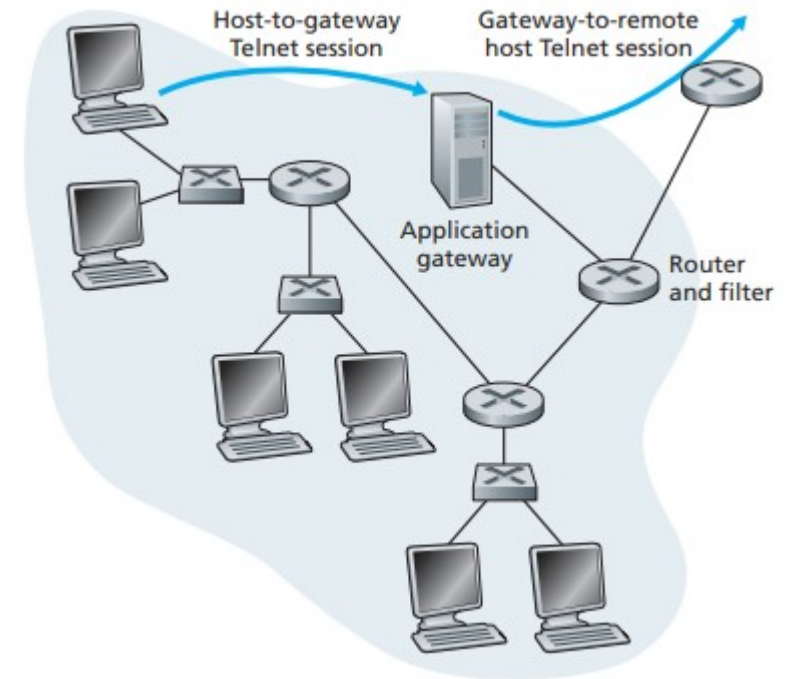
- Above filtering methods mostly check IP/ports, but it could be useful **to allow/deny functionality depending on users or application**, for example:
  - Only **technicians could be allowed to use certain protocols** (e.g., Telnet with outside world) that are denied for normal users.
  - Only **specific types of messages or commands** are allowed within a protocol.
- Such task is beyond the capabilities of traditional and stateful filters as information about **the identity of the internal users is application-layer data** and is not included in the IP/TCP/UDP headers.
- An **application gateway or application-level gateway (AGL)** allow us to filter packets depending on application-layer data. These are typically **implemented as a separated server that works in combination with the firewall**.



# Network Security

## Firewall: Application Gateway

- The general idea is to **deny all connection of a specific protocol** (e.g., Telnet) **except those that are from/to the application gateway**.
- If a user wants to use the application gateway (and then the restricted protocol) **he/she must log to it** (user ID and password).
  - Here the **server checks if the user has the permission** for that protocol.
  - If so, **all requests/responses are forwarded through the application gateway** (proxy).
- Internal **networks often have multiple application gateways** for different protocols (e.g., Telnet, HTTP, FTP, e-mail, etc.).
- There are some disadvantages:
  - A **different application gateway is needed for each application**.
  - **Performance penalty** due to proxying (especially with many users).
  - Software on **host-devices must know how to use the application gateway**.



# Network Security

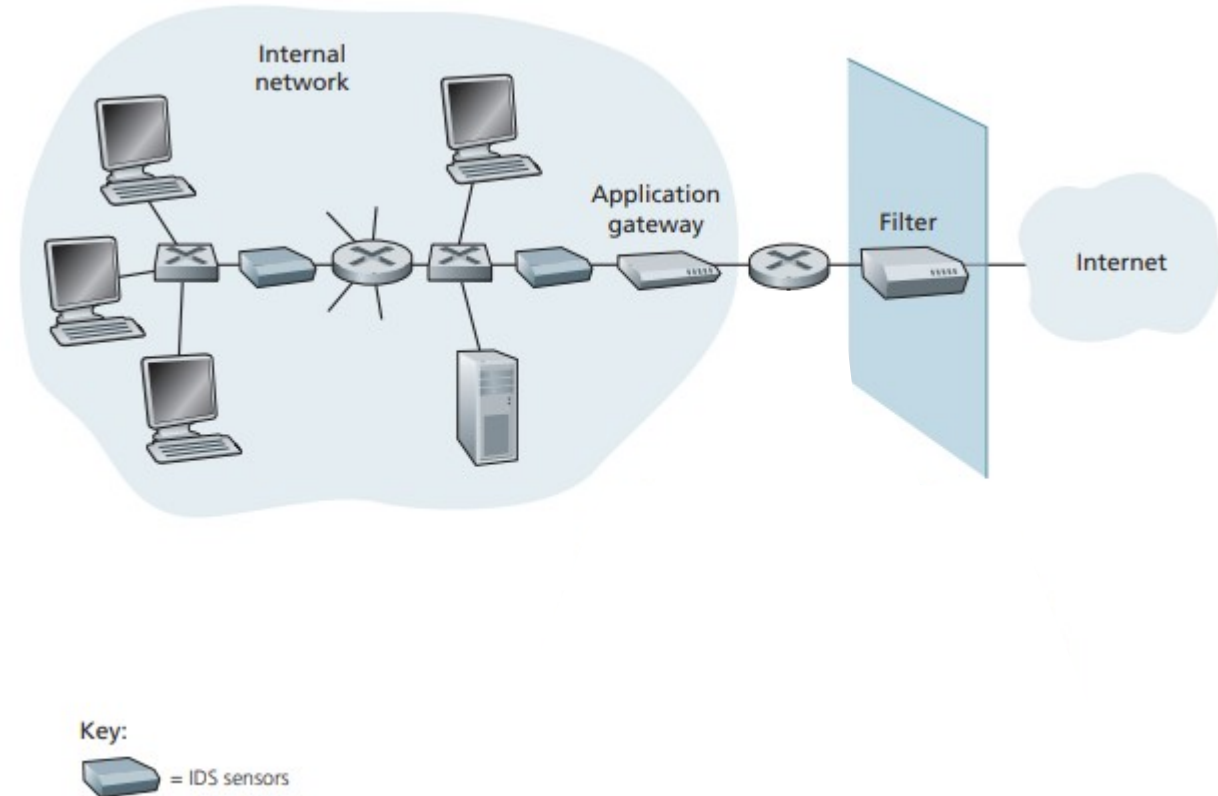
## Intrusion Detection Systems

- In order to detect some attack types, we may need to perform **deep packet inspection** by checking for known attack patterns or suspicious traffic.
- The **Intrusion Detection System (IDS)** is a group of specialized devices that monitor the network, looking for suspicious packets.
  - The **primary role of the IDS is to recognize** potentially malicious packets **and to alert** the network administrator (the administrator is then in charge of doing something).
  - In addition, **it is also possible to block** (i.e., filter out) potentially malicious packets. In this case, we may refer to it as an **Intrusion Prevention System (IPS)**.
- Organizations rely on IDS to **detect a wide range of attacks** (or of potential attacks):
  - Network or port scan (e.g., nmap).
  - DoS flooding.
  - Worms and viruses.
  - ...

# Network Security

## Intrusion Detection Systems

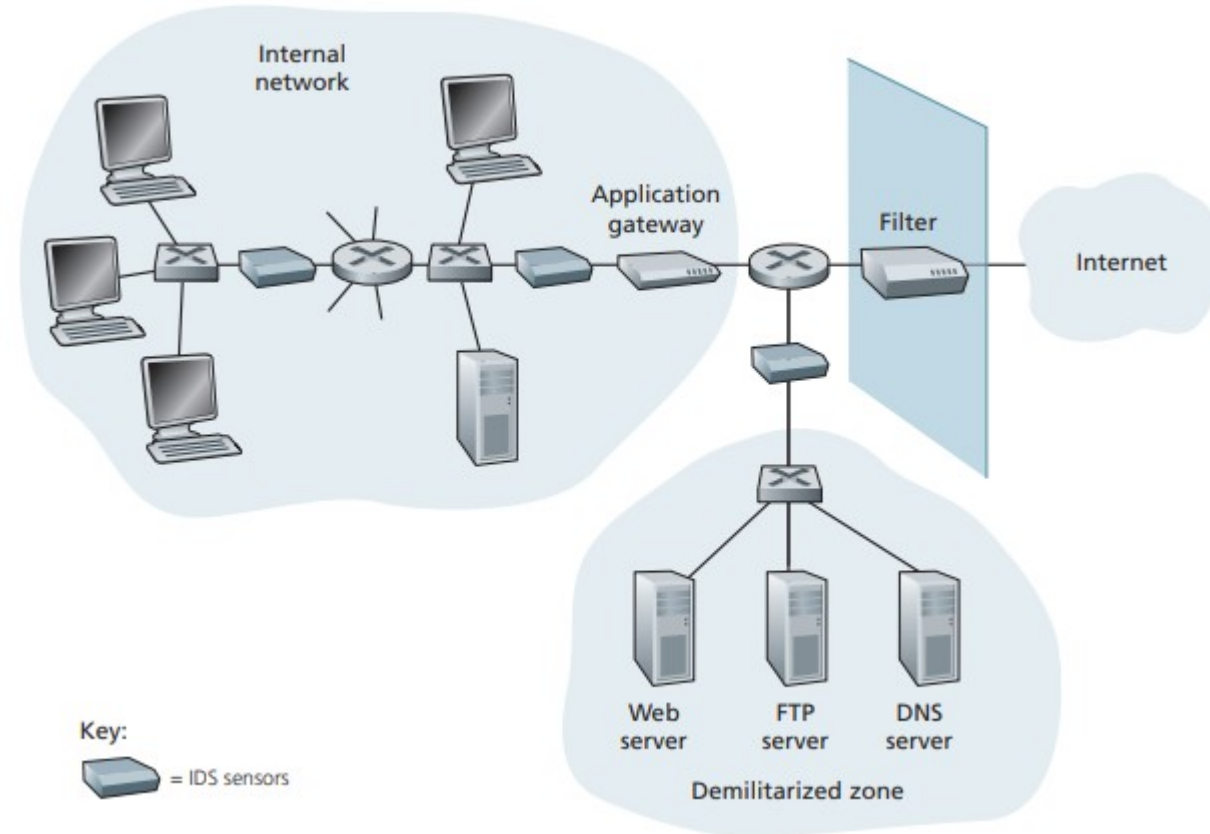
- The system is often composed by:
  - One or more **IDS sensors**.
  - A single central **IDS processor** that collects and integrates the information and sends the alarms.
- IDS can be:
  - **Signature-based**: maintains an extensive database of attack signatures, i.e., a **set of rules related to an intrusion activity** (most common).
  - **Anomaly-based**: creates a traffic profile and checks for streams that are **statistically unusual**.



# Network Security

## Demilitarized Zone

- In large networks there is often the problem of **having 2 different levels of security** for different devices.
  - For example, **servers that need to communicate with the outside world** could use a soft filtering.
- A **demilitarized zone (DMZ)** is **low-security region** in which restrictions are limited and servers can be hosted.
- A typical approach is to put the **DMZ in between 2 firewalls**: an outer (less restrictive) and an inner (more restrictive) one.



Example of DMZ within an IDS:

- The **high-security region** is protected by a packet filter and an application gateway and monitored by IDS sensors.
- The **low-security region** (the DMZ) is only protected by the filter and monitored by the sensors.